

SOP Forge - Voice Triage Engine

Comprehensive Project Documentation

1. Project Overview

The **SOP Forge Voice Engine** is a full-duplex, real-time AI voice triage application designed to act as a primary receptionist or switchboard for an enterprise. It listens to user queries via voice, evaluates their intent using a sophisticated LLM routing agent, and determines the most appropriate department to transfer the call to (e.g., IT Support, Human Resources, Finance).

2. Architecture & Technology Stack

The project adopts a decoupled client-server architecture, allowing the frontend to be highly responsive while the backend handles intensive AI and audio processing.

2.1 Frontend (Voice Client)

- Framework:** Next.js (React)
- Language:** TypeScript (.tsx)
- Styling:** Vanilla CSS Modules (page.module.css) with deep mobile responsiveness and custom micro-animations.
- Audio Capture:** Browser MediaRecorder API streaming audio/webm; codecs=opus chunks.
- Animation Engine:** GSAP (GreenSock Animation Platform) is used to power the real-time audio visualizer bars, responding directly to microphone byte frequency data via the AudioContext API.
- Local TTS:** Browser window.speechSynthesis is utilized specifically for the initial greeting to achieve zero-latency onboarding before handing off to the backend.
- Networking:** Native WebSockets (ws:// / wss://) for uninterrupted, bi-directional audio and JSON event streaming.

2.2 Backend (Voice Engine & Routing)

- Framework:** FastAPI (Python)
- Concurrency:** asyncio and httpx for non-blocking HTTP requests and WebSockets.
- Speech-to-Text (STT):** Groq API utilizing the whisper-large-v3-turbo model. It features auto-language detection, enabling users to speak in English, Urdu, or Hindi seamlessly.
- Text-to-Speech (TTS):** ElevenLabs API (eleven_turbo_v2_5). The backend streams raw audio bytes back to the frontend in real-time as they are generated, rather than waiting for the entire audio file to process.
- Intent Routing Agent:** Powered by LangGraph and Groq (llama-3.1-70b-versatile). The agent evaluates transcribed text against internal Standard Operating Procedures (SOPs) to route the call to:
 - General_HR
 - Accounts_Finance
 - IT_Support
 - No_Transfer (for general conversation)

2.3 Database

- Provider:** Supabase (PostgreSQL)
- Schema:** Contains tables for logging call metadata, transcripts, and routing decisions for analytical review (defined in schema.sql).

3. Core Workflows & Logic

3.1 The Audio Loop

- Initiation:** The user clicks the start button. The frontend requests microphone permissions and uses window.speechSynthesis to deliver an instant greeting.
- VAD (Voice Activity Detection):** The frontend constantly monitors microphone input. To save bandwidth and prevent echo, it only transmits audio chunks over the WebSocket when the user is actively speaking and the AI is silent.
- STT Processing:** The FastAPI backend receives the binary Opus audio, decodes it, and sends it to Groq Whisper.
- Hallucination Filtering:** A custom regex-based filter evaluates the transcript to silently drop known Whisper AI hallucinations (e.g., "Thank you.", "[Music]").
- LLM Routing:** The valid transcript is passed to the LangGraph agent. The agent decides whether to converse naturally or trigger a department_transfer event.
- TTS Streaming:** The AI's text response is sent to ElevenLabs, and the resulting audio bytes are streamed back over the WebSocket to the frontend for immediate playback.

3.2 Bug Fixes & Edge Cases Handled

- Garbage Collection Bug:** Fixed a notorious Chrome/Safari bug where the SpeechSynthesis.onend event fails to fire by attaching the utterance object to the global window scope.
- TTS Bleed:** Implemented an 800ms cooldown window after the AI speaks to prevent the microphone from capturing the tail-end of the AI's voice and looping it back to the STT engine.

4. Deployment Strategy

The decoupled architecture requires a split deployment:

- Frontend Deployment:** Hosted on **Vercel**. Configured via standard Next.js build presets. Environment variables are injected at build time (NEXT_PUBLIC_WS_URL).
- Backend Deployment:** Hosted via **Cloudflare Tunnels** or **Render** (via Docker). Because Vercel Serverless Functions do not support long-lived WebSocket connections, the FastAPI server runs independently and binds to port 8000. Cloudflare tunnels provide a secure, SSL-encrypted (wss://) pipeline directly to the backend.